Name : S.Bala Yokesh
Class : IT Batch 2
Roll no : 2025115080

DS Homework Set – 6

1. Simulate the undo and redo option in a word package using a doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
  char text[100];
  struct Node *prev, *next;
};

struct Node *current = NULL;

void save(char text[]) {
  struct Node *n = malloc(sizeof(struct Node));
  strcpy(n->text, text);
  n->prev = current;
  n->next = NULL;

  if (current != NULL) {
    current->next = n;
  }
  current = n;
}

void undo() {
  if (current != NULL && current->prev != NULL)
    current = current->prev;
  else {
    printf("Nothing to undo\n");
    return;
  }
  printf("Current text: %s\n", current->text);
}

void redo() {
  if (current != NULL && current->next != NULL)
    current = current->next;
  else {
    printf("Nothing to redo\n");
    return;
  }
  printf("Current text: %s\n", current->text);
}

int main() {
```

```
    save("Hello");
    save("Hello World");
    save("Hello World!");

    printf("Current text: %s\n", current->text);
    undo();
    undo();
    redo();

    return 0;
}
```

Output ;

```
Current text: Hello World!
Current text: Hello World
Current text: Hello
Current text: Hello World
```

2. Given two sorted lists L1 and L2, write a procedure to compute L1 ∩ L2 and L1 ∪ L2 using basic list operations.

```
#include <stdio.h>

int main() {
    int a[] = {1, 3, 5, 7};
    int b[] = {3, 5, 6, 8};
    int n = 4, m = 4;
    int i = 0, j = 0;

    printf("Intersection: ");

    i = 0;
    j = 0;

    while (i < n && j < m) {
        if (a[i] == b[j]) {
            printf("%d ", a[i]);
            i++;
            j++;
        }
        else if (a[i] < b[j])
            i++;
        else
            j++;
    }

    printf("\nUnion: ");

    i = 0;
    j = 0;

    while (i < n && j < m) {
```

```
    if (a[i] < b[j])
      printf("%d ", a[i++]);
    else if (a[i] > b[j])
      printf("%d ", b[j++]);
    else {
      printf("%d ", a[i]);
      i++;
      j++;
    }
  }

  while (i < n)
    printf("%d ", a[i++]);

  while (j < m)
    printf("%d ", b[j++]);

  printf("\n");
  return 0;
}
```

Output ;

Intersection: 3 5
Union: 1 3 5 6 7 8

3. Swap two elements by adjusting only the pointers (and not the data) using doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->next = NULL;
  n->prev = NULL;

  if (head == NULL) {
    head = n;
    return;
  }

  struct Node *t = head;
  while (t->next != NULL)
    t = t->next;
```

```
  t->next = n;
  n->prev = t;
}

void swapNodes(struct Node *x, struct Node *y) {
  if (x == y || x == NULL || y == NULL)
    return;

  struct Node *t = head;
  while (t != NULL && t != x && t != y)
    t = t->next;

  if (t == NULL)
    return;

  t = head;
  while (t != NULL && t != x && t != y)
    t = t->next;

  t = head;
  while (t != NULL && t != x)
    t = t->next;

  if (t == NULL) {
    struct Node *temp = x;
    x = y;
    y = temp;
  }

  if (y->next == x) {
    struct Node *temp = x;
    x = y;
    y = temp;
  }

  if (x->next == y) {
    struct Node *p = x->prev;
    struct Node *q = y->next;

    if (p) p->next = y;
    else head = y;

    if (q) q->prev = x;

    x->next = q;
    x->prev = y;
    y->prev = p;
    y->next = x;
    return;
  }

  struct Node *xp = x->prev, *xn = x->next;
  struct Node *yp = y->prev, *yn = y->next;
```

```
  if (xp) xp->next = y;
  else head = y;

  if (xn) xn->prev = y;

  if (yp) yp->next = x;
  else head = x;

  if (yn) yn->prev = x;

  x->prev = yp;
  x->next = yn;
  y->prev = xp;
  y->next = xn;
}

void display() {
  struct Node *t = head;
  while (t != NULL) {
    printf("%d ", t->data);
    t = t->next;
  }
  printf("\n");
}

int main() {
  insert(10);
  insert(20);
  insert(30);
  insert(40);

  printf("Before swapping: ");
  display();

  swapNodes(head, head->next->next);

  printf("After swapping: ");
  display();

  return 0;
}
```

Output :

Before swapping: 10 20 30 40
After swapping: 30 20 10 40

4. Write a C program to reverse a given self-adjusting list. A self-adjusting list is like a regular list, except that all insertions are performed at the front and when an element is accessed by a find, it is moved to the front of the list without changing the relative order of other elements.

```
#include <stdio.h>
```

```c
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

struct Node *head = NULL;

void insertFront(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->next = head;
  head = n;
}

void find(int key) {
  struct Node *t = head, *prev = NULL;

  while (t != NULL && t->data != key) {
    prev = t;
    t = t->next;
  }

  if (t == NULL) {
    printf("Element not found\n");
    return;
  }

  if (prev != NULL) {
    prev->next = t->next;
    t->next = head;
    head = t;
  }
}

void reverse() {
  struct Node *prev = NULL, *cur = head, *next;

  while (cur != NULL) {
    next = cur->next;
    cur->next = prev;
    prev = cur;
    cur = next;
  }

  head = prev;
}

void display() {
  struct Node *t = head;
  while (t != NULL) {
    printf("%d ", t->data);
```

```
    t = t->next;
  }
  printf("\n");
}

int main() {
  insertFront(30);
  insertFront(20);
  insertFront(10);

  printf("Original list: ");
  display();

  find(30);
  printf("After find(30): ");
  display();

  reverse();
  printf("Reversed list: ");
  display();

  return 0;
}
```

Output :

```
Original list: 10 20 30
After find(30): 30 10 20
Reversed list: 20 10 30
```

5. Write a C program to reverse a given doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->next = NULL;
  n->prev = NULL;

  if (head == NULL) {
    head = n;
    return;
  }
```

```
  struct Node *t = head;
  while (t->next != NULL)
    t = t->next;

  t->next = n;
  n->prev = t;
}

void reverse() {
  struct Node *cur = head, *temp = NULL;

  while (cur != NULL) {
    temp = cur->prev;
    cur->prev = cur->next;
    cur->next = temp;
    cur = cur->prev;
  }

  if (temp != NULL)
    head = temp->prev;
}

void display() {
  struct Node *t = head;
  while (t != NULL) {
    printf("%d ", t->data);
    t = t->next;
  }
  printf("\n");
}

int main() {
  insert(10);
  insert(20);
  insert(30);
  insert(40);

  printf("Before reversing: ");
  display();

  reverse();

  printf("After reversing: ");
  display();

  return 0;
}
```

Output :

Before reversing: 10 20 30 40
After reversing: 40 30 20 10

6. Write a C program to delete the middle element of a DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->next = NULL;
  n->prev = NULL;

  if (head == NULL) {
    head = n;
    return;
  }

  struct Node *t = head;
  while (t->next != NULL)
    t = t->next;

  t->next = n;
  n->prev = t;
}

void deleteMiddle() {
  if (head == NULL)
    return;

  struct Node *slow = head, *fast = head;

  while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;
  }

  if (slow->prev != NULL)
    slow->prev->next = slow->next;
  else
    head = slow->next;

  if (slow->next != NULL)
    slow->next->prev = slow->prev;

  free(slow);
}
```

```
void display() {
   struct Node *t = head;
   while (t != NULL) {
      printf("%d ", t->data);
      t = t->next;
   }
   printf("\n");
}

int main() {
   insert(10);
   insert(20);
   insert(30);
   insert(40);
   insert(50);

   printf("Before deletion: ");
   display();

   deleteMiddle();

   printf("After deletion: ");
   display();

   return 0;
}
```

Output :

```
Before deletion: 10 20 30 40 50
After deletion: 10 20 40 50
```

7. Remove duplicate elements from a (i) sorted DLL (ii) unsorted DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *prev, *next;
};

struct Node *create(int x) {
   struct Node *n = malloc(sizeof(struct Node));
   n->data = x;
   n->prev = n->next = NULL;
   return n;
}

void insert(struct Node **head, int x) {
   struct Node *n = create(x);

   if (*head == NULL) {
```

```
    *head = n;
    return;
  }

  struct Node *t = *head;
  while (t->next != NULL)
    t = t->next;

  t->next = n;
  n->prev = t;
}

void removeSortedDuplicates(struct Node *head) {
  struct Node *t = head;

  while (t != NULL && t->next != NULL) {
    if (t->data == t->next->data) {
      struct Node *d = t->next;
      t->next = d->next;

      if (d->next != NULL)
        d->next->prev = t;

      free(d);
    } else {
      t = t->next;
    }
  }
}

void removeUnsortedDuplicates(struct Node *head) {
  struct Node *t = head;

  while (t != NULL) {
    struct Node *p = t->next;

    while (p != NULL) {
      if (p->data == t->data) {
        struct Node *d = p;
        p = p->next;

        if (d->prev != NULL)
          d->prev->next = d->next;

        if (d->next != NULL)
          d->next->prev = d->prev;

        free(d);
      } else {
        p = p->next;
      }
    }
```

```
      t = t->next;
    }
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node *sorted = NULL;
    struct Node *unsorted = NULL;

    insert(&sorted, 10);
    insert(&sorted, 10);
    insert(&sorted, 20);
    insert(&sorted, 30);
    insert(&sorted, 30);

    insert(&unsorted, 10);
    insert(&unsorted, 20);
    insert(&unsorted, 10);
    insert(&unsorted, 30);
    insert(&unsorted, 20);

    removeSortedDuplicates(sorted);
    removeUnsortedDuplicates(unsorted);

    printf("Sorted DLL: ");
    display(sorted);

    printf("Unsorted DLL: ");
    display(unsorted);

    return 0;
}
```

Output :

```
Sorted DLL: 10 20 30
Unsorted DLL: 10 20 30
```

8. Calculate the frequency count of a given integer in a circular DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
```

```
};

struct Node *head = NULL;

void insert(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;

  if (head == NULL) {
    head = n;
    n->next = n;
    n->prev = n;
    return;
  }

  struct Node *last = head->prev;

  n->next = head;
  n->prev = last;
  last->next = n;
  head->prev = n;
}

int frequency(int key) {
  if (head == NULL)
    return 0;

  int count = 0;
  struct Node *t = head;

  do {
    if (t->data == key)
      count++;

    t = t->next;
  } while (t != head);

  return count;
}

int main() {
  insert(10);
  insert(20);
  insert(10);
  insert(30);
  insert(10);

  printf("Frequency of 10 = %d\n", frequency(10));

  return 0;
}
```

Output :

Frequency of 10 = 3

9. Search for a particular element in a binary tree.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node *create(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = NULL;
  return n;
}

int search(struct Node *root, int key) {
  if (root == NULL)
    return 0;

  if (root->data == key)
    return 1;

  return search(root->left, key) ||
      search(root->right, key);
}

int main() {
  struct Node *root = create(10);
  root->left = create(20);
  root->right = create(30);
  root->left->left = create(40);
  root->left->right = create(50);

  int key = 50;

  if (search(root, key))
    printf("%d found in the tree\n", key);
  else
    printf("%d not found in the tree\n", key);

  return 0;
}
```

Output :

50 found in the tree

10. Calculate the height or depth of a tree and print it.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node *create(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = NULL;
  return n;
}

int height(struct Node *root) {
  if (root == NULL)
    return 0;

  int lh = height(root->left);
  int rh = height(root->right);

  return 1 + (lh > rh ? lh : rh);
}

int main() {
  struct Node *root = create(1);
  root->left = create(2);
  root->right = create(3);
  root->left->left = create(4);
  root->left->right = create(5);

  printf("Height of tree = %d nodes\n", height(root));

  return 0;
}
```

Output :

```
Height of tree = 3 nodes
```

11. Given a binary tree, convert this to a doubly linked list keeping the same order (inorder traversal).

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node *head = NULL, *prevNode = NULL;
```

```
struct Node *create(int x) {
   struct Node *n = malloc(sizeof(struct Node));
   n->data = x;
   n->left = n->right = NULL;
   return n;
}

void convert(struct Node *root) {
   if (root == NULL)
      return;

   convert(root->left);

   if (prevNode == NULL)
      head = root;
   else {
      prevNode->right = root;
      root->left = prevNode;
   }

   prevNode = root;

   convert(root->right);
}

void display() {
   struct Node *t = head;

   while (t != NULL) {
      printf("%d ", t->data);
      t = t->right;
   }
   printf("\n");
}

int main() {
   struct Node *root = create(4);
   root->left = create(2);
   root->right = create(6);
   root->left->left = create(1);
   root->left->right = create(3);
   root->right->left = create(5);
   root->right->right = create(7);

   convert(root);

   printf("DLL in inorder: ");
   display();

   return 0;
}
```

Output ;

DLL in inorder: 1 2 3 4 5 6 7

12. Given a binary tree, the task is to connect all nodes that are at the same level using next right pointer.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right, *nextRight;
};

struct Node *create(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = n->nextRight = NULL;
  return n;
}

void connect(struct Node *root) {
  if (root == NULL)
    return;

  struct Node *q[100];
  int front = 0, rear = 0;

  q[rear++] = root;

  while (front < rear) {
    int count = rear - front;
    struct Node *prev = NULL;

    for (int i = 0; i < count; i++) {
      struct Node *cur = q[front++];

      if (prev != NULL)
        prev->nextRight = cur;

      prev = cur;

      if (cur->left != NULL)
        q[rear++] = cur->left;

      if (cur->right != NULL)
        q[rear++] = cur->right;
    }

    prev->nextRight = NULL;
  }
}

int main() {
  struct Node *root = create(1);
```

```
  root->left = create(2);
  root->right = create(3);
  root->left->left = create(4);
  root->left->right = create(5);
  root->right->right = create(6);

  connect(root);

  printf("Level 1: %d -> NULL\n", root->data);

  printf("Level 2: %d -> %d -> NULL\n",
      root->left->data, root->left->nextRight->data);

  printf("Level 3: %d -> %d -> %d -> NULL\n",
      root->left->left->data,
      root->left->left->nextRight->data,
      root->left->left->nextRight->nextRight->data);

  return 0;
}
```

Output ;

```
Level 1: 1 -> NULL
Level 2: 2 -> 3 -> NULL
Level 3: 4 -> 5 -> 6 -> NULL
```

13. Implement priority queue using binary search tree and perform operations.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node *create(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = NULL;
  return n;
}

struct Node *insert(struct Node *root, int x) {
  if (root == NULL)
    return create(x);

  if (x < root->data)
    root->left = insert(root->left, x);
  else if (x > root->data)
    root->right = insert(root->right, x);
```

```
  return root;
}

struct Node *findMax(struct Node *root) {
  if (root == NULL)
    return NULL;

  while (root->right != NULL)
    root = root->right;

  return root;
}

struct Node *deleteMax(struct Node *root) {
  if (root == NULL)
    return NULL;

  if (root->right == NULL) {
    struct Node *temp = root->left;
    free(root);
    return temp;
  }

  root->right = deleteMax(root->right);
  return root;
}

void display(struct Node *root) {
  if (root == NULL)
    return;

  display(root->right);
  printf("%d ", root->data);
  display(root->left);
}

int main() {
  struct Node *root = NULL;

  root = insert(root, 30);
  root = insert(root, 50);
  root = insert(root, 20);
  root = insert(root, 40);
  root = insert(root, 10);

  printf("Priority order: ");
  display(root);

  struct Node *max = findMax(root);
  if (max != NULL)
    printf("\nHighest priority: %d\n", max->data);

  root = deleteMax(root);
```

```
  printf("After deleting highest priority: ");
  display(root);
  printf("\n");

  return 0;
}
```

Output :

```
Priority order: 50 40 30 20 10
Highest priority: 50
After deleting highest priority: 40 30 20 10
```

14. In a given binary search tree write routines to delete a node with no children, one child and two children.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node *create(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = NULL;
  return n;
}

struct Node *insert(struct Node *root, int x) {
  if (root == NULL)
    return create(x);

  if (x < root->data)
    root->left = insert(root->left, x);
  else if (x > root->data)
    root->right = insert(root->right, x);

  return root;
}

struct Node *minNode(struct Node *root) {
  while (root != NULL && root->left != NULL)
    root = root->left;

  return root;
}

struct Node *deleteNode(struct Node *root, int key) {
  if (root == NULL)
```

```
    return NULL;

  if (key < root->data)
    root->left = deleteNode(root->left, key);
  else if (key > root->data)
    root->right = deleteNode(root->right, key);
  else {
    // Case 1 and Case 2: zero or one child
    if (root->left == NULL) {
      struct Node *temp = root->right;
      free(root);
      return temp;
    }

    if (root->right == NULL) {
      struct Node *temp = root->left;
      free(root);
      return temp;
    }

    // Case 3: two children
    struct Node *temp = minNode(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
  }

  return root;
}

void inorder(struct Node *root) {
  if (root == NULL)
    return;

  inorder(root->left);
  printf("%d ", root->data);
  inorder(root->right);
}

int main() {
  struct Node *root = NULL;

  root = insert(root, 50);
  root = insert(root, 30);
  root = insert(root, 70);
  root = insert(root, 20);
  root = insert(root, 40);
  root = insert(root, 60);
  root = insert(root, 80);

  printf("Original BST: ");
  inorder(root);

  root = deleteNode(root, 20);
```

```
  printf("\nAfter deleting leaf 20: ");
  inorder(root);

  root = deleteNode(root, 30);
  printf("\nAfter deleting node 30: ");
  inorder(root);

  root = deleteNode(root, 50);
  printf("\nAfter deleting node 50: ");
  inorder(root);

  printf("\n");
  return 0;
}
```

Output :

```
Original BST: 20 30 40 50 60 70 80
After deleting leaf 20: 30 40 50 60 70 80
After deleting node 30: 40 50 60 70 80
After deleting node 50: 40 60 70 80
```

15. Convert the given postfix expression to expression tree.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node {
  char data;
  struct Node *left, *right;
};

struct Node *create(char x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = NULL;
  return n;
}

void inorder(struct Node *root) {
  if (root == NULL)
    return;

  if (root->left != NULL || root->right != NULL)
    printf("(");

  inorder(root->left);
  printf("%c", root->data);
  inorder(root->right);

  if (root->left != NULL || root->right != NULL)
    printf(")");
```

```
}

int main() {
  char postfix[] = "ab+c*";
  struct Node *stack[100];
  int top = -1;

  for (int i = 0; postfix[i] != '\0'; i++) {
    char ch = postfix[i];

    if (isalnum((unsigned char)ch))
      stack[++top] = create(ch);
    else {
      struct Node *n = create(ch);
      n->right = stack[top--];
      n->left = stack[top--];
      stack[++top] = n;
    }
  }

  printf("Expression: ");
  inorder(stack[top]);
  printf("\n");

  return 0;
}
```

Output :

```
Expression: ((a+b)*c)
```

16. Construct binary search tree using (i) inorder (ii) preorder and (iii) postorder.

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node *create(int x) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = x;
  n->left = n->right = NULL;
  return n;
}

struct Node *insert(struct Node *root, int x) {
  if (root == NULL)
    return create(x);
```

```
  if (x < root->data)
    root->left = insert(root->left, x);
  else if (x > root->data)
    root->right = insert(root->right, x);

  return root;
}

void inorder(struct Node *root) {
  if (root == NULL)
    return;

  inorder(root->left);
  printf("%d ", root->data);
  inorder(root->right);
}

struct Node *buildPre(int a[], int *i, int n,
            long long low, long long high) {
  if (*i >= n || a[*i] <= low || a[*i] >= high)
    return NULL;

  int x = a[(*i)++];
  struct Node *root = create(x);

  root->left = buildPre(a, i, n, low, x);
  root->right = buildPre(a, i, n, x, high);

  return root;
}

struct Node *buildPost(int a[], int *i,
             long long low, long long high) {
  if (*i < 0 || a[*i] <= low || a[*i] >= high)
    return NULL;

  int x = a[(*i)--];
  struct Node *root = create(x);

  root->right = buildPost(a, i, low, x);
  root->left = buildPost(a, i, x, high);

  return root;
}

int main() {
  int pre[] = {40, 20, 10, 30, 60, 50, 70};
  int post[] = {10, 30, 20, 50, 70, 60, 40};
  int n = 7, i;

  struct Node *root1 = NULL;
  for (i = 0; i < n; i++)
    root1 = insert(root1, pre[i]);
```

```
  printf("BST from preorder: ");
  inorder(root1);

  i = 0;
  struct Node *root2 =
    buildPre(pre, &i, n, LLONG_MIN, LLONG_MAX);

  printf("\nConstructed from preorder: ");
  inorder(root2);

  i = n - 1;
  struct Node *root3 =
    buildPost(post, &i, LLONG_MIN, LLONG_MAX);

  printf("\nConstructed from postorder: ");
  inorder(root3);

  printf("\n");
  return 0;
}
```

Output :

```
BST from preorder: 10 20 30 40 50 60 70
Constructed from preorder: 10 20 30 40 50 60 70
Constructed from postorder: 10 20 30 40 50 60 70
```

17. Implement heap sort for a set of numbers.

```
#include <stdio.h>

void heapify(int a[], int n, int i) {
  int largest = i;
  int left = 2 * i + 1;
  int right = 2 * i + 2;

  if (left < n && a[left] > a[largest])
    largest = left;

  if (right < n && a[right] > a[largest])
    largest = right;

  if (largest != i) {
    int temp = a[i];
    a[i] = a[largest];
    a[largest] = temp;

    heapify(a, n, largest);
  }
}

void heapSort(int a[], int n) {
```

```
  for (int i = n / 2 - 1; i >= 0; i--)
    heapify(a, n, i);

  for (int i = n - 1; i > 0; i--) {
    int temp = a[0];
    a[0] = a[i];
    a[i] = temp;

    heapify(a, i, 0);
  }
}

int main() {
  int a[] = {50, 20, 30, 25, 15, 22, 32};
  int n = sizeof(a) / sizeof(a[0]);

  printf("Original array: ");
  for (int i = 0; i < n; i++)
    printf("%d ", a[i]);

  heapSort(a, n);

  printf("\nSorted array: ");
  for (int i = 0; i < n; i++)
    printf("%d ", a[i]);

  printf("\n");
  return 0;
}
```

Output :

Original array: 50 20 30 25 15 22 32
Sorted array: 15 20 22 25 30 32 50